

UNIVERSIDADE PAULISTA
TURMA DE CIÊNCIA DA COMPUTAÇÃO

LETÍCIA DE JESUS AMORIM - F35FJC0
JOÃO LUCAS ANTUNES MODENA – R096AD4
MATEUS RUIZ – G7985I2

Atividade Prática Supervisionada (APS)
Arquitetura de Redes de Computadores

SÃO PAULO 2026

1. Indice

1. Indice 2
2. Objetivo e Motivação 3
    2.1. Objetivo 3
    2.2. Motivação 4
3. Introdução 4
4. Fundamentos da comunicação de dados em rede 6
5. Plano de desenvolvimento da aplicação 12
6. Projeto 18
7. Relatório e Código 24
    7.1. Relatório e Código – Server 24
    7.2. Relatório e Código – Client 30
8. Apresentação do programa 34
    8.1. Inicialização do Servidor 34
    8.2. Inicialização do Cliente 35
    8.3 Exemplo de Comunicação 36
Referências bibliográficas 30
Ficha APS 40

2. Objetivo e Motivação

2.1 Objetivo

O presente trabalho tem como objetivo desenvolver uma aplicação de comunicação em rede utilizando a linguagem de programação C# juntamente com o protocolo TCP/IP, permitindo a troca de mensagens em tempo real entre múltiplos usuários conectados em uma mesma rede local. A proposta busca aplicar, de forma prática e objetiva, os conceitos estudados ao longo da disciplina relacionados à comunicação de dados, arquitetura central-servidor, programação orientada a objetos e desenvolvimento de aplicações distribuídas.

A aplicação desenvolvida simula um sistema de comunicação utilizado pela Secretaria de Estado do Meio Ambiente, órgão responsável pelo monitoramento das atividades industriais que contribuem para a poluição do Rio Tietê. Nesse cenário, inspetores ambientais posicionados em diferentes indústrias precisam transmitir informações continuamente para uma central responsável pela supervisão, análise e armazenamento dos dados coletados durante as inspeções realizadas.

Com base nessa necessidade, foi criado um sistema de troca de mensagens capaz de estabelecer comunicação eficiente entre os usuários conectados. O sistema utiliza sockets TCP/IP para garantir uma conexão estável e confiável entre servidor e centrais, permitindo o envio e recebimento de mensagens em tempo real. O servidor atua como elemento central da aplicação, sendo responsável por gerenciar as conexões dos usuários, controlar o fluxo de mensagens e manter a comunicação ativa entre todos os participantes conectados à rede.

As centrais, por sua vez, representam os inspetores ambientais espalhados pelas diferentes indústrias monitoradas. Cada central pode enviar informações importantes para a central principal, como relatórios de emissão de poluentes, alertas ambientais e atualizações sobre irregularidades encontradas durante as inspeções. Além disso, o sistema permite que as mensagens sejam distribuídas de maneira rápida e organizada, garantindo maior eficiência na comunicação entre os envolvidos.

Outro objetivo importante deste trabalho é proporcionar experiência prática no desenvolvimento de aplicações de rede utilizando recursos da linguagem C#. Durante a implementação do sistema, foram aplicados conhecimentos relacionados à criação e manipulação de sockets, gerenciamento de conexões simultâneas através de threads, tratamento de exceções e sincronização de processos. Esses conceitos são fundamentais para o desenvolvimento de sistemas modernos que dependem de comunicação em rede e processamento simultâneo de informações.

Além do aspecto técnico, o projeto também busca demonstrar a importância da tecnologia no apoio às atividades de monitoramento ambiental. A utilização de sistemas de comunicação em tempo real pode contribuir significativamente para a agilidade na tomada de decisões, permitindo respostas mais rápidas diante de situações que possam causar impactos ambientais.

Dessa forma, o trabalho não apenas atende aos objetivos acadêmicos da disciplina, mas também evidencia como os conceitos de redes de computadores podem ser aplicados em situações reais, oferecendo soluções eficientes para problemas de comunicação e gerenciamento de informações em ambientes distribuídos.

2.2 Motivação

A necessidade de comunicação rápida e eficiente entre diferentes setores torna os sistemas em rede fundamentais em diversas áreas da sociedade. Atualmente, aplicações distribuídas permitem a troca instantânea de informações entre usuários e organizações localizadas em diferentes lugares.

No contexto ambiental, tecnologias de monitoramento e comunicação em tempo real são importantes para acompanhar atividades potencialmente poluentes. O Rio Tietê, um dos principais rios do estado de São Paulo, enfrenta há muitos anos problemas relacionados à poluição causada por indústrias, descarte irregular de resíduos e crescimento urbano. Assim, torna-se necessário o uso de ferramentas que auxiliem os órgãos responsáveis no monitoramento dessas atividades.

A motivação deste trabalho surgiu da oportunidade de aplicar, na prática, os conhecimentos adquiridos na disciplina de redes e programação. A proposta de desenvolver uma ferramenta de comunicação para a Secretaria de Estado do Meio Ambiente permitiu unir conceitos de computação com uma questão ambiental relevante.

Além disso, o projeto possibilitou compreender melhor o funcionamento da arquitetura cliente-servidor e dos protocolos TCP/IP, bem como utilizar recursos da linguagem C#, como sockets, threads e tratamento de exceções.

Por fim, este trabalho demonstra como a tecnologia pode contribuir para melhorar a comunicação, agilizar o compartilhamento de informações e apoiar ações de fiscalização e preservação ambiental.

3. Introdução

Nas últimas décadas, o avanço das tecnologias da informação e comunicação transformou significativamente a forma como pessoas, empresas e instituições

realizam suas atividades. A troca de informações passou a ocorrer de maneira rápida graças ao desenvolvimento das redes de computadores e da internet. Atualmente, sistemas conectados em rede estão presentes em diversos setores, como educação, saúde, indústria, comércio e monitoramento ambiental.

As redes de computadores desempenham papel fundamental no funcionamento de sistemas modernos, permitindo o compartilhamento de dados, recursos e serviços entre diferentes dispositivos conectados. Além disso, possibilitam comunicação rápida e eficiente entre usuários localizados em diferentes regiões geográficas. Essa conectividade tornou-se indispensável para organizações que dependem de informações em tempo real para tomada de decisões, gerenciamento de processos e coordenação de equipes distribuídas.

Com o crescimento das redes, surgiram protocolos responsáveis por padronizar a comunicação entre dispositivos. Entre os mais utilizados destaca-se o TCP/IP (Transmission Control Protocol/Internet Protocol), considerado a base da comunicação na internet e em redes locais.

O protocolo TCP/IP é amplamente utilizado devido à sua confiabilidade e eficiência na transmissão de dados entre clientes e servidores. O TCP garante que as informações sejam entregues corretamente, enquanto o IP realiza o endereçamento e roteamento dos dados entre os dispositivos conectados.

Outro conceito importante é o modelo cliente-servidor. Nesse modelo, o servidor gerencia conexões e disponibiliza serviços aos clientes conectados, enquanto os clientes enviam solicitações e recebem informações. Essa arquitetura é utilizada em sistemas de mensagens, aplicações web e diversos serviços digitais.

Além da relevância tecnológica, os sistemas de comunicação em rede possuem grande importância em atividades que exigem monitoramento constante e troca rápida de informações. Empresas privadas, órgãos governamentais e instituições ambientais utilizam ferramentas computacionais para melhorar a comunicação entre equipes, agilizar processos internos e aumentar a eficiência operacional. Em cenários que envolvem fiscalização e monitoramento ambiental, por exemplo, a comunicação eficiente pode contribuir diretamente para a prevenção de danos ambientais e para a rápida identificação de irregularidades.

No contexto ambiental, o monitoramento de atividades industriais é essencial, principalmente em regiões afetadas pela poluição. O Rio Tietê, por exemplo, sofre há muitos anos com problemas relacionados ao despejo irregular de resíduos industriais e esgoto. Dessa forma, órgãos ambientais necessitam de sistemas eficientes para acompanhar as atividades das indústrias.

Considerando esse cenário, o presente trabalho propõe o desenvolvimento de uma aplicação de comunicação em rede utilizando a linguagem C# e o protocolo TCP/IP. O sistema permite a troca de mensagens em tempo real entre inspetores ambientais e uma central responsável pelo monitoramento das informações coletadas durante as inspeções.

A aplicação foi desenvolvida utilizando a arquitetura cliente-servidor, na qual um servidor central aceita conexões de múltiplos clientes, gerencia os usuários conectados e distribui as mensagens recebidas. Essa abordagem permite comunicação contínua entre os participantes da rede.

O cliente desenvolvido possibilita o envio e recebimento de mensagens em tempo real através de sockets TCP/IP. Os sockets são mecanismos fundamentais para comunicação entre aplicações em rede, permitindo a troca de dados entre dispositivos conectados.

Durante o desenvolvimento do projeto, foram aplicados conhecimentos relacionados à programação orientada a objetos, utilização de sockets, manipulação de streams de dados, gerenciamento de conexões e implementação de processamento concorrente através de threads.

Outro aspecto importante foi a utilização de threads para permitir múltiplas conexões simultâneas no servidor. Essa abordagem garante que vários clientes possam permanecer conectados ao mesmo tempo sem comprometer o funcionamento da aplicação.

Além disso, o projeto envolveu tratamento de exceções e gerenciamento de erros relacionados às conexões de rede, contribuindo para maior estabilidade e confiabilidade do sistema desenvolvido.

A escolha da linguagem C# ocorreu devido à sua ampla utilização no desenvolvimento de aplicações desktop e sistemas corporativos, além de oferecer recursos eficientes para programação orientada a objetos e comunicação em rede.

Portanto, este trabalho busca demonstrar, de forma prática, os conceitos relacionados às redes de computadores e protocolos de comunicação estudados ao longo da disciplina. A aplicação desenvolvida representa uma solução simples e funcional para troca de mensagens em tempo real, destacando a importância das tecnologias de comunicação em atividades de monitoramento ambiental.

4. Fundamentos Da Comunicação De Dados Em Rede

A comunicação de dados em rede consiste no processo de transmissão, recepção e compartilhamento de informações entre dois ou mais dispositivos

conectados através de um meio físico ou sem fio. Esse processo é essencial para o funcionamento dos sistemas computacionais modernos, permitindo que computadores, servidores, dispositivos móveis, sensores, equipamentos industriais e diversos outros dispositivos troquem informações de maneira rápida, organizada e eficiente.

Atualmente, praticamente todas as atividades digitais dependem de algum tipo de comunicação em rede. O acesso à internet, o envio de mensagens instantâneas, chamadas de vídeo, armazenamento em nuvem, sistemas bancários, plataformas educacionais e aplicações corporativas utilizam mecanismos de transmissão de dados para funcionar corretamente. Dessa forma, compreender os fundamentos da comunicação de dados é indispensável para o desenvolvimento de sistemas modernos e para o entendimento da infraestrutura tecnológica utilizada no mundo atual.

As redes de computadores podem ser definidas como conjuntos de dispositivos interconectados capazes de compartilhar informações, recursos e serviços. Esses dispositivos podem incluir computadores pessoais, notebooks, smartphones, impressoras, servidores, roteadores e equipamentos especializados. O principal objetivo das redes é permitir a comunicação entre esses dispositivos de forma eficiente e segura.

As redes podem ser classificadas de acordo com sua abrangência geográfica. As redes locais, conhecidas como LAN (Local Area Network), são utilizadas em ambientes menores, como residências, escolas, laboratórios e empresas. Nessas redes, os dispositivos geralmente estão conectados em um mesmo espaço físico e possuem alta velocidade de comunicação.

As redes metropolitanas, chamadas de MAN (Metropolitan Area Network), abrangem áreas maiores, como cidades ou regiões urbanas. Já as redes WAN (Wide Area Network) possuem alcance ainda maior, podendo conectar dispositivos localizados em diferentes estados ou países. A internet é considerada o maior exemplo de rede WAN existente atualmente.

Além da classificação por alcance, as redes também podem ser classificadas quanto à topologia utilizada. A topologia representa a forma como os dispositivos estão organizados e conectados entre si. Entre as principais topologias destacam-se a topologia em estrela, barramento, anel e malha.

Na topologia em estrela, todos os dispositivos estão conectados a um equipamento central, geralmente um switch ou roteador. Esse modelo é amplamente utilizado devido à facilidade de gerenciamento e manutenção. Já na topologia em barramento, todos os dispositivos compartilham um único meio de transmissão. Embora seja uma estrutura simples, apresenta limitações relacionadas ao desempenho e à confiabilidade.

A topologia em anel conecta os dispositivos em formato circular, permitindo que os dados circulem entre os equipamentos até alcançar o destino correto. Já a topologia em malha oferece múltiplos caminhos de comunicação entre os dispositivos, aumentando a redundância e a tolerância a falhas.

O funcionamento das redes depende da existência de protocolos de comunicação. Protocolos são conjuntos de regras responsáveis por definir como os dados serão transmitidos entre os dispositivos conectados. Eles estabelecem padrões relacionados ao formato das mensagens, sincronização, controle de erros, autenticação e gerenciamento das conexões.

Sem protocolos, os dispositivos não conseguiriam interpretar corretamente as informações recebidas. Dessa forma, os protocolos garantem que diferentes equipamentos e sistemas operacionais possam se comunicar de maneira padronizada.

Entre os protocolos mais importantes destaca-se o TCP/IP (Transmission Control Protocol/Internet Protocol), considerado o principal padrão utilizado nas redes modernas e na internet. O modelo TCP/IP é composto por diferentes camadas, sendo cada uma responsável por funções específicas relacionadas à comunicação de dados.

A camada de aplicação é responsável pela interação direta com os programas utilizados pelos usuários. Nessa camada atuam protocolos como HTTP, HTTPS, FTP, SMTP e DNS. A camada de transporte é responsável pelo controle da comunicação entre origem e destino, utilizando protocolos como TCP e UDP.

A camada de internet realiza o endereçamento e roteamento dos pacotes através do protocolo IP. Já a camada de acesso à rede é responsável pela transmissão física dos dados através dos meios de comunicação disponíveis.

O protocolo IP possui papel fundamental na comunicação em rede. Ele é responsável pelo endereçamento lógico dos dispositivos conectados. Cada equipamento conectado à rede recebe um endereço IP único, utilizado para identificação durante a transmissão dos dados.

Existem duas versões principais do protocolo IP: IPv4 e IPv6. O IPv4 utiliza endereços de 32 bits, representados geralmente em formato decimal, como por exemplo 192.168.0.1. Entretanto, devido ao crescimento da internet e ao aumento da quantidade de dispositivos conectados, surgiu a necessidade de criação do IPv6.

O IPv6 utiliza endereços de 128 bits, oferecendo uma quantidade muito maior de combinações possíveis. Isso permite suportar bilhões de dispositivos conectados simultaneamente, garantindo maior escalabilidade para as redes modernas.

Além do endereçamento, o protocolo IP também é responsável pelo roteamento dos pacotes de dados. Os roteadores analisam os endereços IP de origem e destino para determinar o melhor caminho que os dados devem percorrer até chegar ao dispositivo correto.

Já o protocolo TCP é responsável por garantir a confiabilidade da comunicação. Ele realiza o controle da transmissão dos dados, verificando se os pacotes chegaram corretamente ao destino. Caso ocorra perda de informações durante o envio, o TCP realiza automaticamente o reenvio dos pacotes perdidos.

O TCP também organiza os dados recebidos na ordem correta, garantindo integridade e consistência das informações transmitidas. Esse protocolo é amplamente utilizado em aplicações que exigem alta confiabilidade, como sistemas bancários, aplicações web e transferência de arquivos.

A comunicação utilizando TCP ocorre através do estabelecimento de uma conexão entre cliente e servidor. Antes da troca de dados, é realizado um processo conhecido como handshake. Esse procedimento estabelece uma conexão sincronizada entre os dispositivos envolvidos, garantindo que ambos estejam preparados para iniciar a comunicação.

Outro protocolo importante é o UDP (User Datagram Protocol). Diferentemente do TCP, o UDP não realiza controle de confirmação ou reenvio de pacotes. Isso torna a comunicação mais rápida, porém menos confiável. O UDP é utilizado em aplicações que priorizam velocidade, como transmissões de vídeo, jogos online e chamadas em tempo real.

Outro conceito importante relacionado às redes é a arquitetura cliente-servidor. Nesse modelo, existe um servidor central responsável por fornecer serviços ou recursos, enquanto os clientes realizam solicitações e recebem respostas do servidor.

Esse modelo é amplamente utilizado em sistemas modernos, incluindo aplicações web, bancos de dados, plataformas corporativas, sistemas de mensagens e serviços em nuvem. O servidor geralmente possui maior capacidade de processamento e armazenamento, sendo responsável pelo gerenciamento centralizado das informações.

No sistema desenvolvido neste trabalho, o servidor é responsável por aceitar conexões de múltiplos clientes, armazenar os usuários conectados e distribuir as mensagens recebidas para os demais participantes da rede. Os clientes representam os inspetores responsáveis pelo envio das informações ambientais para a central de monitoramento.

Para permitir a comunicação entre aplicações em rede, são utilizados sockets. Os sockets funcionam como pontos de comunicação entre dispositivos conectados, permitindo o envio e recebimento de dados através da rede.

Em aplicações TCP/IP, cada socket está associado a um endereço IP e a uma porta específica. Dessa forma, os sockets permitem que diferentes aplicações utilizem simultaneamente a mesma conexão de rede sem interferência entre si.

As portas possuem papel fundamental na comunicação em rede, pois permitem identificar qual serviço ou aplicação deve receber os dados transmitidos. Existem portas padronizadas para diversos serviços conhecidos, como a porta 80 utilizada pelo protocolo HTTP e a porta 443 utilizada pelo HTTPS.

No sistema desenvolvido, foi utilizada a porta 12345 para estabelecer a comunicação entre servidor e clientes. Essa porta foi definida especificamente para o funcionamento da aplicação de monitoramento ambiental.

Outro aspecto importante da comunicação de dados é a utilização de streams de rede. Os streams representam fluxos contínuos de dados transmitidos entre cliente e servidor. Eles permitem leitura e escrita de informações em tempo real durante a comunicação.

No desenvolvimento da aplicação, o NetworkStream foi utilizado para leitura e envio das mensagens trocadas entre os dispositivos conectados. Esse recurso facilita o gerenciamento da transmissão de dados através da rede.

Além da transmissão de dados, aplicações em rede precisam lidar com múltiplas conexões simultâneas. Para resolver esse problema, é comum utilizar técnicas de processamento concorrente, como threads.

Threads são linhas de execução independentes dentro de um programa. Elas permitem que diferentes tarefas sejam executadas simultaneamente, melhorando o desempenho e a eficiência da aplicação.

No sistema desenvolvido, cada cliente conectado ao servidor é tratado através de uma thread separada. Isso permite múltiplas conexões simultâneas sem interromper o funcionamento geral do sistema.

O gerenciamento de múltiplos clientes é essencial em sistemas de comunicação em tempo real. Sem o uso de threads, o servidor conseguiria atender apenas um cliente por vez, tornando a comunicação limitada e ineficiente.

Outro conceito relevante é o broadcast de mensagens. O broadcast consiste no envio de uma informação para todos os clientes conectados à rede. No projeto desenvolvido, quando um cliente envia uma mensagem ao servidor, essa mensagem é retransmitida para os demais usuários conectados, permitindo comunicação coletiva em tempo real.

Além disso, sistemas de comunicação em rede precisam possuir mecanismos de tratamento de erros e exceções. Problemas como perda de conexão, falhas na transmissão de dados e encerramento inesperado de clientes podem ocorrer durante a execução da aplicação.

Por isso, o tratamento de exceções é importante para evitar falhas no sistema e garantir maior estabilidade durante a comunicação. Técnicas de reconexão automática e monitoramento de conexões também podem ser utilizadas para aumentar a confiabilidade da aplicação.

Outro elemento fundamental nas redes de computadores é a codificação dos dados transmitidos. As mensagens precisam ser convertidas em bytes antes de serem enviadas pela rede. No sistema desenvolvido, foi utilizada a codificação UTF-8 para converter as mensagens em dados binários, permitindo a transmissão correta dos caracteres.

A segurança também é um fator extremamente importante na comunicação de dados em rede. Em aplicações reais, mecanismos adicionais de autenticação, criptografia e controle de acesso são utilizados para proteger as informações transmitidas.

A criptografia permite transformar os dados em formatos ilegíveis para usuários não autorizados. Protocolos seguros como HTTPS utilizam certificados digitais e criptografia SSL/TLS para proteger a comunicação entre cliente e servidor.

Outro aspecto importante relacionado à segurança é o controle de acesso. Sistemas modernos frequentemente utilizam autenticação por login e senha, tokens de acesso e permissões específicas para garantir que apenas usuários autorizados possam acessar determinadas informações.

As redes de computadores possuem grande importância no mundo atual devido à necessidade constante de compartilhamento de informações. Empresas, instituições governamentais, hospitais, escolas, universidades e indústrias dependem de sistemas conectados para execução de suas atividades diárias.

No setor industrial, redes são utilizadas para automação de processos e monitoramento de equipamentos. Na área da saúde, permitem compartilhamento rápido de prontuários médicos e exames. Na educação, possibilitam ensino remoto e acesso a plataformas digitais.

No contexto ambiental, sistemas de comunicação em tempo real podem auxiliar órgãos responsáveis pelo monitoramento e fiscalização. A transmissão rápida de informações permite respostas mais ágeis diante de situações de risco ambiental, contribuindo para melhor controle das atividades monitoradas.

Sensores conectados em rede podem coletar dados relacionados à qualidade do ar, temperatura, umidade, níveis de poluição e condições climáticas. Essas

informações podem ser enviadas automaticamente para centrais de monitoramento, permitindo análise em tempo real.

Dessa forma, os conceitos de comunicação de dados em rede são fundamentais para o desenvolvimento de aplicações modernas. O estudo de protocolos, arquitetura cliente-servidor, sockets, threads, segurança e transmissão de dados permite compreender como sistemas distribuídos funcionam e como podem ser aplicados em diferentes áreas da sociedade.

Com o avanço constante da tecnologia e o crescimento da internet das coisas (IoT), da computação em nuvem e das aplicações distribuídas, a comunicação em rede torna-se cada vez mais importante. O domínio desses conceitos é essencial para profissionais da área de tecnologia da informação e para o desenvolvimento de soluções inovadoras capazes de atender às necessidades da sociedade moderna.

5. Plano De Desenvolvimento Da Aplicação

O desenvolvimento da aplicação foi realizado com o objetivo de criar uma ferramenta simples e funcional de comunicação em rede utilizando a linguagem C# juntamente com o protocolo TCP/IP. A aplicação foi baseada no modelo cliente-servidor, permitindo a troca de mensagens em tempo real entre múltiplos usuários conectados em uma mesma rede.

O sistema foi projetado para atender à proposta apresentada pela Secretaria de Estado do Meio Ambiente, permitindo a comunicação entre inspetores ambientais espalhados em diferentes indústrias e uma central responsável pelo monitoramento das informações coletadas.

O planejamento do desenvolvimento ocorreu em etapas, desde a análise inicial da proposta até os testes finais de funcionamento da aplicação.

Levantamento de Requisitos

A primeira etapa do desenvolvimento consistiu na análise da proposta do trabalho e levantamento dos requisitos necessários para funcionamento do sistema.

Foi identificado que a aplicação deveria permitir:

- Comunicação em tempo real entre usuários;
- Conexão simultânea de múltiplos clientes;
- Troca de mensagens através da rede;
- Gerenciamento centralizado das conexões;

- Estabilidade durante o envio e recebimento de dados;
- Utilização do protocolo TCP/IP.

Além dos requisitos funcionais, também foram considerados requisitos não funcionais relacionados ao desempenho, simplicidade de utilização e estabilidade da comunicação.

Outro ponto importante analisado foi a necessidade de utilização de processamento concorrente, permitindo que vários clientes permanecessem conectados simultaneamente ao servidor sem interromper o funcionamento da aplicação.

Escolha da Linguagem e Tecnologias

Após a análise inicial, foi definida a utilização da linguagem C# para desenvolvimento do sistema. A escolha ocorreu devido à facilidade oferecida pela linguagem para desenvolvimento de aplicações orientadas a objetos e comunicação em rede.

A plataforma .NET também oferece bibliotecas prontas para manipulação de sockets TCP/IP, gerenciamento de threads e transmissão de dados através de streams, facilitando significativamente a implementação da aplicação.

As principais tecnologias utilizadas foram:

- Linguagem C#;
- Plataforma .NET;
- Protocolo TCP/IP;
- Sockets TCP;
- Threads;
- NetworkStream.

A utilização dessas tecnologias permitiu o desenvolvimento de uma aplicação simples, organizada e funcional.

Definição da Arquitetura do Sistema

Após a definição das tecnologias, foi escolhida a arquitetura cliente-servidor para o funcionamento da aplicação.

Nesse modelo, o servidor possui a responsabilidade de:

- Aceitar conexões dos clientes;

- Gerenciar usuários conectados;
- Receber mensagens enviadas;
- Distribuir mensagens para os demais clientes;
- Manter a comunicação ativa.

Os clientes possuem a função de:

- Conectar-se ao servidor;
- Enviar mensagens;
- Receber mensagens transmitidas;
- Manter comunicação contínua com os demais usuários.

Essa arquitetura foi escolhida devido à sua simplicidade e ampla utilização em sistemas de comunicação distribuídos.

Desenvolvimento do Servidor

O desenvolvimento do servidor foi uma das etapas mais importantes do projeto. O servidor foi implementado utilizando a classe TcpListener, responsável por aguardar conexões de clientes através de uma porta específica.

Inicialmente foi definida a porta 12345 para realização da comunicação entre os dispositivos conectados à rede.

Após inicializar o servidor, o sistema permanece continuamente aguardando novas conexões. Quando um cliente se conecta, o servidor adiciona esse cliente em uma lista responsável pelo armazenamento das conexões ativas.

Para permitir múltiplos clientes simultaneamente, foi utilizada uma thread separada para cada conexão realizada. Dessa forma, cada cliente possui um fluxo independente de comunicação com o servidor.

O servidor também possui um método responsável por realizar o broadcast das mensagens. Quando um cliente envia uma mensagem, o servidor recebe os dados e retransmite a informação para os demais clientes conectados.

Essa abordagem permite comunicação coletiva em tempo real entre todos os participantes da aplicação.

Além disso, foram implementados mecanismos básicos de tratamento de exceções para evitar falhas durante desconexões inesperadas dos clientes.

Desenvolvimento do Cliente

Após o desenvolvimento do servidor, foi iniciada a implementação da aplicação cliente.

O cliente foi desenvolvido utilizando a classe TcpClient, responsável por estabelecer conexão com o servidor através do endereço IP e da porta definida.

No projeto desenvolvido, foi utilizado o endereço IP local 127.0.0.1 para testes da aplicação na própria máquina.

Após estabelecer conexão com o servidor, o cliente cria um fluxo de comunicação utilizando NetworkStream. Esse fluxo permite envio e recebimento contínuo de mensagens.

Para garantir comunicação simultânea, também foi utilizada uma thread dedicada ao recebimento das mensagens enviadas pelo servidor. Isso permite que o usuário continue digitando mensagens enquanto novas informações chegam em tempo real.

As mensagens digitadas pelo usuário são convertidas para bytes utilizando codificação UTF-8 e enviadas através do stream de rede.

O cliente permanece continuamente executando até o encerramento manual da aplicação ou perda da conexão com o servidor.

Implementação das Threads

Um dos principais recursos utilizados no desenvolvimento foi a implementação de threads.

As threads permitem execução simultânea de diferentes tarefas dentro do sistema. Sem sua utilização, o servidor conseguiria atender apenas um cliente por vez.

No servidor, cada cliente conectado recebe uma thread específica responsável por:

- Receber mensagens;
- Monitorar a conexão;
- Detectar desconexões;
- Encaminhar mensagens recebidas.

No cliente, uma thread adicional foi utilizada para escutar continuamente as mensagens recebidas do servidor.

A utilização desse recurso tornou o sistema mais eficiente e permitiu comunicação em tempo real entre múltiplos usuários.

Gerenciamento das Conexões

Outro aspecto importante do desenvolvimento foi o gerenciamento das conexões ativas.

O servidor mantém uma lista contendo todos os clientes conectados no momento da execução. Essa lista é utilizada para envio das mensagens através do método de broadcast.

Quando um cliente se desconecta ou ocorre alguma falha na conexão, o sistema identifica o problema através do tratamento de exceções.

Embora o projeto possua uma estrutura simples, o gerenciamento das conexões foi suficiente para garantir funcionamento adequado da aplicação durante os testes realizados.

Tratamento de Erros e Exceções

Durante o desenvolvimento foram implementados tratamentos básicos de exceções utilizando estruturas try-catch.

O objetivo foi evitar que falhas de conexão encerrassem completamente o sistema.

Problemas como:

- Desconexão inesperada;
- Encerramento do cliente;
- Falha na leitura de dados;
- Interrupção da comunicação;

foram tratados para manter maior estabilidade da aplicação.

O tratamento de erros é essencial em sistemas de comunicação em rede, pois conexões podem falhar por diversos motivos relacionados à infraestrutura da rede ou comportamento dos usuários.

Testes Realizados

Após a conclusão da implementação, foram realizados testes para verificar o funcionamento da aplicação.

Os principais testes executados foram:

- Teste de conexão entre cliente e servidor;
- Teste de envio de mensagens;

- Teste de recebimento simultâneo;
- Teste de múltiplos clientes conectados;
- Teste de desconexão;
- Teste de estabilidade da aplicação.

Os testes demonstraram que o sistema conseguiu estabelecer comunicação corretamente entre os dispositivos conectados.

As mensagens foram transmitidas em tempo real e distribuídas corretamente para os clientes participantes da comunicação.

Também foi possível verificar o funcionamento das threads e do gerenciamento simultâneo das conexões.

Limitações da Aplicação

Apesar do funcionamento correto, o sistema desenvolvido apresenta algumas limitações devido à simplicidade do projeto.

Entre as principais limitações estão:

- Ausência de interface gráfica;
- Falta de autenticação de usuários;
- Ausência de criptografia das mensagens;
- Não armazenamento do histórico das conversas;
- Utilização apenas em rede local;
- Ausência de transferência de arquivos.

Mesmo com essas limitações, o sistema atende aos objetivos propostos pela atividade prática supervisionada.

Possíveis Melhorias Futuras

O sistema desenvolvido pode receber diversas melhorias futuras para aumentar funcionalidades e segurança da aplicação.

Entre as possíveis melhorias destacam-se:

- Desenvolvimento de interface gráfica;
- Implementação de login e senha;
- Criptografia das mensagens;
- Armazenamento de conversas em banco de dados;

- Envio de arquivos;
- Comunicação por áudio e vídeo;
- Comunicação multicast;
- Integração com serviços em nuvem;
- Sistema de notificações.

Essas melhorias tornariam a aplicação mais próxima de sistemas profissionais de comunicação utilizados atualmente.

Considerações Finais do Desenvolvimento

O desenvolvimento da aplicação permitiu aplicar na prática diversos conceitos estudados ao longo da disciplina, principalmente relacionados à comunicação de dados em rede, arquitetura cliente-servidor e programação concorrente.

A utilização da linguagem C# juntamente com os protocolos TCP/IP demonstrou ser eficiente para criação de aplicações simples de comunicação em rede.

Além do aprendizado técnico, o projeto também demonstrou como sistemas computacionais podem auxiliar processos de monitoramento ambiental e troca de informações em tempo real.

Dessa forma, o desenvolvimento da aplicação atingiu os objetivos propostos, resultando em um sistema funcional capaz de estabelecer comunicação entre múltiplos usuários conectados em rede.

6. **PROJETO**

O projeto desenvolvido foi estruturado utilizando a arquitetura cliente-servidor, modelo amplamente utilizado em sistemas de comunicação em rede. Essa estrutura foi escolhida devido à sua simplicidade, eficiência e facilidade de gerenciamento das conexões entre os dispositivos participantes da comunicação.

A aplicação foi dividida em dois módulos principais:

- Módulo Servidor;
- Módulo Cliente.

Cada módulo possui responsabilidades específicas dentro do sistema, garantindo organização do código, separação das funções e melhor gerenciamento da comunicação entre os usuários conectados.

O sistema foi desenvolvido utilizando a linguagem C# juntamente com recursos da plataforma .NET voltados para comunicação em rede, processamento concorrente e manipulação de conexões TCP/IP.

Estrutura Geral do Sistema

A estrutura geral da aplicação é composta por um servidor central responsável pelo gerenciamento das conexões e diversos clientes responsáveis pela troca de mensagens.

O funcionamento ocorre da seguinte forma:

1. O servidor é iniciado e permanece aguardando conexões;
2. Os clientes realizam conexão através do endereço IP e da porta definida;
3. O servidor aceita os clientes conectados;
4. Cada cliente recebe uma thread específica para comunicação;
5. As mensagens enviadas pelos clientes são recebidas pelo servidor;
6. O servidor retransmite as mensagens para os demais usuários conectados.

Essa estrutura permite comunicação contínua e simultânea entre múltiplos participantes conectados à rede.

Módulo Servidor

O módulo servidor representa o núcleo principal da aplicação. Ele é responsável por controlar toda a comunicação entre os clientes conectados.

As principais funções do servidor são:

- Inicializar o serviço de comunicação;
- Aguardar conexões de clientes;
- Gerenciar múltiplas conexões simultâneas;
- Receber mensagens enviadas;
- Distribuir mensagens para os clientes;
- Monitorar desconexões;
- Manter a estabilidade da aplicação.

O servidor foi implementado utilizando a classe TcpListener, responsável por criar um ponto de escuta na rede através de uma porta específica.

No sistema desenvolvido, foi utilizada a porta 12345 para realização da comunicação.

Após sua inicialização, o servidor permanece continuamente aguardando solicitações de conexão. Quando um cliente se conecta, a conexão é aceita e adicionada em uma lista responsável pelo armazenamento dos clientes ativos.

Gerenciamento de Clientes

O gerenciamento dos clientes conectados é realizado através de uma lista do tipo List.

Essa lista possui a função de:

- Armazenar os clientes conectados;
- Permitir envio de mensagens para múltiplos usuários;
- Controlar conexões ativas;
- Facilitar o broadcast das mensagens.

Sempre que um novo cliente realiza conexão, ele é automaticamente adicionado à lista de clientes ativos.

Caso ocorra desconexão, o sistema identifica a falha através do tratamento de exceções.

Threads do Servidor

Para permitir múltiplas conexões simultâneas, o servidor utiliza threads independentes para cada cliente conectado.

Cada thread possui responsabilidade de:

- Receber mensagens do cliente;
- Monitorar o estado da conexão;
- Encaminhar mensagens recebidas;
- Detectar desconexões inesperadas.

Essa abordagem permite que diversos clientes permaneçam conectados simultaneamente sem bloquear o funcionamento geral do sistema.

Sem a utilização de threads, o servidor conseguiria atender apenas um cliente por vez, limitando significativamente a comunicação.

Método de Broadcast

O sistema utiliza um método de broadcast responsável pelo envio das mensagens para todos os clientes conectados.

Quando um usuário envia uma mensagem:

1. O servidor recebe os dados;
2. A mensagem é convertida para texto;
3. O sistema percorre a lista de clientes;
4. A mensagem é retransmitida para os demais usuários.

Esse mecanismo permite comunicação coletiva em tempo real.

Módulo Cliente

O módulo cliente representa os usuários responsáveis pela troca de mensagens dentro do sistema.

No contexto do projeto, os clientes simulam os inspetores ambientais responsáveis pelo envio de informações relacionadas ao monitoramento das indústrias.

As principais funções do cliente são:

- Conectar-se ao servidor;
- Enviar mensagens;
- Receber mensagens em tempo real;
- Manter conexão ativa;
- Exibir informações recebidas.

O cliente foi implementado utilizando a classe TcpClient, responsável por estabelecer comunicação TCP/IP com o servidor.

Após iniciar a aplicação, o cliente realiza conexão utilizando o endereço IP do servidor e a porta previamente definida.

Comunicação Cliente-Servidor

A comunicação entre cliente e servidor ocorre através de sockets TCP/IP.

Os sockets funcionam como canais de comunicação responsáveis pela troca de dados entre os dispositivos conectados.

Após estabelecer conexão, o cliente cria um fluxo de comunicação utilizando NetworkStream.

Esse fluxo permite:

- Envio de dados;
- Recebimento de mensagens;
- Comunicação contínua;
- Troca de informações em tempo real.

As mensagens digitadas pelos usuários são convertidas em bytes utilizando codificação UTF-8 antes de serem transmitidas pela rede.

Thread de Recebimento do Cliente

Para permitir envio e recebimento simultâneo de mensagens, o cliente também utiliza uma thread específica para escutar continuamente os dados enviados pelo servidor.

Essa thread permanece executando paralelamente ao sistema principal.

Dessa forma, o usuário consegue:

- Digitar mensagens;
- Receber informações simultaneamente;
- Manter comunicação contínua.

Esse recurso melhora significativamente a experiência de utilização da aplicação.

Estrutura das Mensagens

As mensagens transmitidas pelo sistema possuem estrutura simples baseada em texto.

O fluxo de transmissão ocorre da seguinte maneira:

1. O usuário digita uma mensagem;
2. A mensagem é convertida para bytes;
3. Os dados são enviados ao servidor;
4. O servidor recebe a informação;
5. A mensagem é retransmitida para os demais clientes;
6. Os clientes recebem e exibem o conteúdo.

Esse modelo simplifica a comunicação e reduz a complexidade da implementação.

Tratamento de Exceções

Durante o desenvolvimento do sistema, foram implementados mecanismos básicos de tratamento de erros utilizando estruturas try-catch.

O tratamento de exceções foi utilizado para:

- Detectar falhas de conexão;
- Evitar encerramento inesperado da aplicação;
- Tratar desconexões de clientes;
- Garantir estabilidade do sistema.

Caso ocorra alguma falha durante a leitura ou envio dos dados, o sistema identifica o problema sem interromper completamente a execução da aplicação.

Organização do Código

O código da aplicação foi estruturado de forma simples e organizada, separando as responsabilidades principais do sistema.

As funções foram divididas em métodos específicos, como:

- Inicialização do servidor;
- Aceitação de clientes;
- Gerenciamento das conexões;
- Recebimento de mensagens;
- Broadcast das informações.

Essa organização facilita manutenção, leitura e futuras melhorias do sistema.

Fluxo de Funcionamento da Aplicação

O funcionamento geral da aplicação pode ser representado da seguinte forma:

Fluxo do Servidor

1. Inicialização do servidor;
2. Definição da porta de comunicação;
3. Espera por conexões;
4. Aceitação de clientes;
5. Criação de threads;

6. Recebimento das mensagens;
7. Broadcast para os clientes conectados.

Fluxo do Cliente

1. Inicialização do cliente;
2. Conexão com o servidor;
3. Criação da thread de recebimento;
4. Digitação de mensagens;
5. Envio de dados;
6. Recebimento das mensagens retransmitidas.

Possíveis Expansões da Estrutura

Embora o sistema desenvolvido possua estrutura simples, ele pode ser expandido futuramente através da criação de novos módulos.

Entre os módulos que poderiam ser implementados estão:

- Interface gráfica;
- Sistema de login;
- Criptografia de mensagens;
- Banco de dados;
- Envio de arquivos;
- Comunicação multicast;
- Registro de histórico;
- Painel administrativo;
- Sistema de notificações.

A modularização da aplicação facilita futuras implementações sem necessidade de reconstrução completa do sistema.

Considerações Finais sobre a Estrutura

A estrutura desenvolvida permitiu a criação de um sistema funcional de comunicação em rede utilizando conceitos fundamentais de programação distribuída e redes de computadores.

A separação entre cliente e servidor, juntamente com o uso de sockets TCP/IP e threads, possibilitou comunicação simultânea entre múltiplos usuários conectados.

Além de atender aos objetivos acadêmicos da atividade proposta, o projeto proporcionou experiência prática no desenvolvimento de aplicações de rede utilizando a linguagem C# e recursos da plataforma .NET.

Dessa forma, a estrutura do programa demonstrou ser adequada para implementação de um sistema simples de comunicação em tempo real aplicado ao contexto de monitoramento ambiental apresentado na proposta do trabalho.

7. Relatoriodas linhas de Código
   7.1 Servidor

O servidor desenvolvido é responsável pelo gerenciamento central da comunicação entre os clientes conectados à rede. Sua principal função é receber conexões, armazenar os clientes ativos e retransmitir as mensagens recebidas para os demais usuários conectados.

A implementação foi realizada utilizando a linguagem C# juntamente com recursos da plataforma .NET voltados para comunicação em rede.

▼ Servidor

```
using System;
using System.Collections.Generic;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading;

class Program
{
    static TcpListener server;
    static List<TcpClient> clients = new List<TcpClient>();

    static void Main()
    {
        int port = 12345;

        server = new TcpListener(IPAddress.Any, port);
        server.Start();

        Console.WriteLine("Servidor iniciado...");

        while (true)
        {
            TcpClient client = server.AcceptTcpClient();
            clients.Add(client);

            Console.WriteLine("Novo cliente conectado!");

            Thread clientThread = new Thread(() => HandleClient(client));
            clientThread.Start();
        }
    }

    static void HandleClient(TcpClient client)
    {
        NetworkStream stream = client.GetStream();
        byte[] buffer = new byte[1024];

        try
        {
            while (true)
            {
                int bytesRead = stream.Read(buffer, 0, buffer.Length);

                if (bytesRead == 0)
                    break;

                string message = Encoding.UTF8.GetString(buffer, 0, bytesRead);

                Console.WriteLine(message);

                Broadcast(message, client);
```

```
            }
        }
        catch
        {
            Console.WriteLine("Cliente desconectado.");
        }
    }

    static void Broadcast(string message, TcpClient sender)
    {
        byte[] data = Encoding.UTF8.GetBytes(message);

        foreach (TcpClient client in clients)
        {
            if (client != sender)
            {
                try
                {
                    NetworkStream stream = client.GetStream();
                    stream.Write(data, 0, data.Length);
                }
                catch { }
            }
        }
    }
}
```

O funcionamento do servidor inicia com a definição de uma porta de comunicação específica. Essa porta funciona como um ponto de acesso da aplicação na rede, permitindo que os clientes saibam exatamente onde devem se conectar.

Após a configuração da porta, o servidor cria um objeto responsável por escutar continuamente novas solicitações de conexão. Esse processo é realizado utilizando recursos da biblioteca de rede do .NET, permitindo que o sistema permaneça ativo aguardando clientes.

Quando o servidor é iniciado, uma mensagem é exibida no console informando que o sistema está em execução e pronto para receber conexões. Essa mensagem serve como confirmação de que o serviço foi iniciado corretamente.

Exemplo de funcionamento inicial:

- Inicialização do servidor;
- Definição da porta de comunicação;
- Ativação do serviço de escuta;
- Aguardando conexões de clientes.

O servidor permanece em execução contínua durante todo o funcionamento da aplicação. Isso significa que novos usuários podem se conectar a qualquer momento sem a necessidade de reiniciar o sistema.

Sempre que um cliente realiza uma conexão, o servidor:

1. Aceita a solicitação de conexão;
2. Cria um canal de comunicação exclusivo para aquele cliente;
3. Armazena o cliente em uma lista interna;
4. Inicia um processo separado para tratar as mensagens daquele usuário.

Essa lista interna é extremamente importante, pois permite que o servidor saiba quais clientes estão conectados no momento e para quais usuários as mensagens deverão ser retransmitidas.

Além disso, o sistema registra no console informações sobre novas conexões realizadas. Dessa forma, é possível acompanhar em tempo real quais usuários estão entrando no sistema.

Exemplo de mensagem exibida no console:

Novo cliente conectado!

Processamento Simultâneo de Clientes

Um dos principais desafios em aplicações de rede é permitir que vários usuários utilizem o sistema ao mesmo tempo sem causar travamentos ou interrupções.

Para resolver esse problema, o servidor utiliza processamento paralelo por meio de threads. Cada cliente conectado recebe uma execução separada, permitindo que o servidor trate múltiplas conexões simultaneamente.

Na prática, isso significa que:

- Um cliente pode enviar mensagens enquanto outro recebe;
- Novos usuários podem entrar no sistema sem interromper conexões existentes;
- O servidor consegue atender vários clientes ao mesmo tempo.

Esse modelo melhora significativamente o desempenho da aplicação e garante comunicação em tempo real entre os participantes.

O uso de threads também evita que uma conexão bloqueie o funcionamento das demais. Caso um cliente apresente lentidão ou desconecte inesperadamente, os outros usuários continuam utilizando o sistema normalmente.

Comunicação Entre Clientes

Após a conexão ser estabelecida, o servidor permanece monitorando continuamente as mensagens enviadas pelos clientes.

Quando um usuário envia uma mensagem:

1. O cliente transmite os dados ao servidor;
2. O servidor recebe os dados pela conexão TCP;
3. A mensagem é convertida para texto;
4. O servidor retransmite a mensagem para os demais clientes conectados.

Esse processo ocorre de maneira praticamente instantânea, permitindo comunicação em tempo real semelhante a aplicações de chat.

O servidor atua como intermediador central da comunicação. Os clientes não se comunicam diretamente entre si; toda troca de mensagens passa obrigatoriamente pelo servidor.

Esse modelo apresenta diversas vantagens:

- Centralização da comunicação;
- Maior controle das conexões;
- Facilidade de gerenciamento dos usuários;
- Melhor organização do fluxo de mensagens.

Além disso, o sistema pode ser expandido futuramente para incluir funcionalidades adicionais, como:

- Identificação de usuários;
- Criação de salas de conversa;
- Registro de mensagens;
- Sistema de autenticação;
- Criptografia de dados.

Tratamento de Desconexões

Durante o funcionamento da aplicação, pode ocorrer a desconexão de clientes, seja por fechamento da aplicação, perda de conexão ou falhas na rede.

O servidor foi preparado para lidar com essas situações de forma segura. Quando uma desconexão é detectada:

- O cliente é removido da lista de conexões ativas;
- Os recursos utilizados pela conexão são liberados;
- O servidor continua funcionando normalmente para os demais usuários.

Esse tratamento evita erros no sistema e impede que conexões inválidas permaneçam armazenadas na memória.

Tecnologias Utilizadas

O desenvolvimento do servidor utilizou os seguintes recursos e tecnologias:

Tecnologia	Função
C#	Linguagem principal da aplicação
.NET	Plataforma de desenvolvimento
TCP/IP	Protocolo de comunicação em rede
TcpListener	Responsável por aceitar conexões
TcpClient	Representa os clientes conectados
Threads	Processamento simultâneo

Considerações Finais

O projeto desenvolvido demonstra o funcionamento básico de um servidor TCP utilizando a linguagem C# e os recursos da plataforma .NET.

A aplicação foi capaz de:

- Aceitar múltiplas conexões simultaneamente;
- Gerenciar clientes conectados;
- Receber mensagens em tempo real;
- Redistribuir mensagens entre os usuários;
- Manter comunicação contínua utilizando TCP/IP.

O uso de sockets e processamento paralelo permitiu a construção de um sistema funcional de comunicação em rede, aplicando na prática conceitos importantes relacionados à programação distribuída e comunicação cliente-servidor.

Além disso, o desenvolvimento do projeto proporcionou aprendizado sobre:

- Estrutura de aplicações em rede;
- Gerenciamento de conexões TCP;
- Comunicação entre processos;
- Concorrência e paralelismo;
- Manipulação de fluxos de dados em tempo real.

Por fim, o sistema desenvolvido serve como base para aplicações mais avançadas de comunicação em rede, podendo futuramente receber melhorias e novas funcionalidades conforme a necessidade do projeto.

7.2 Client

O código desenvolvido representa a aplicação cliente de um sistema de comunicação em rede utilizando sockets TCP na linguagem C#. O principal objetivo do programa é permitir a troca de mensagens em tempo real entre diferentes usuários conectados um servidor central, simulando o funcionamento básico de um sistema de chat.

▼ Client

```
using System;
using System.Net.Sockets;
using System.Text;
using System.Threading;

class Program
{
    static void Main()
    {
        string serverIP = "127.0.0.1";
        int port = 12345;

        TcpClient client = new TcpClient(serverIP, port);

        Console.WriteLine("Conectado ao servidor!");
        Console.Write("Insira sua estação de controle: ");
        string nome = Console.ReadLine();

        NetworkStream stream = client.GetStream();

        Thread receiveThread = new Thread(() =>
        {
            byte[] buffer = new byte[1024];
```

```
            while (true)
            {
                try
                {
                    int bytesRead = stream.Read(buffer, 0, buffer.Length);

                    if (bytesRead == 0)
                        break;

                    string message = Encoding.UTF8.GetString(buffer, 0, bytesRead

                    Console.WriteLine(message);
                }
                catch
                {
                    break;
                }
            }
        });

        receiveThread.Start();

        while (true)
        {
            string messageText = Console.ReadLine();
            string horario = DateTime.Now.ToString("HH:mm");
            string message = "[" + horario + "] " + nome + ": " + messageText;
```



```
            byte[] data = Encoding.UTF8.GetBytes(message);

            stream.Write(data, 0, data.Length);
        }
    }
}
```

Inicialmente, o programa define o endereço IP do servidor e a porta de comunicação que será utilizada para estabelecer a conexão. No exemplo apresentado, foi utilizado o endereço IP local 127.0.0.1, também conhecido como localhost, que permite testar a comunicação na própria máquina. A porta escolhida foi a 12345, utilizada como canal de comunicação entre cliente e servidor.

Após definir essas informações, o cliente cria um objeto da classe TcpClient, responsável por iniciar a conexão TCP com o servidor. Quando a conexão é realizada com sucesso, o sistema exibe uma mensagem no console informando ao usuário que a comunicação foi estabelecida corretamente.

Em seguida, o programa solicita que o usuário informe o nome da sua estação de controle. Esse nome funciona como identificação do usuário dentro do sistema de mensagens, permitindo que todos os participantes saibam quem enviou cada mensagem recebida no chat.

Depois da identificação do usuário, o cliente obtém o fluxo de comunicação da conexão através do método GetStream(). Esse fluxo, representado pela classe NetworkStream, é responsável pelo envio e recebimento de dados entre cliente e servidor.

Uma das principais características do sistema é a utilização de uma thread separada exclusivamente para o recebimento de mensagens. Essa thread é criada utilizando a classe Thread, permitindo que o programa execute duas tarefas simultaneamente:

- receber mensagens enviadas pelo servidor;
- permitir que o usuário continue digitando novas mensagens.

Essa abordagem é extremamente importante em aplicações de comunicação em tempo real, pois evita que o programa fique bloqueado aguardando apenas uma operação por vez.

Dentro da thread de recebimento, o sistema cria um buffer de bytes com tamanho de 1024 posições. Esse buffer é utilizado para armazenar temporariamente os dados recebidos pela rede. O método Read() é responsável por ler os dados enviados pelo servidor e armazená-los nesse buffer.

Após a leitura dos dados, o programa converte os bytes recebidos para texto utilizando a codificação UTF-8 através da classe Encoding.UTF8. Em seguida, a mensagem é exibida diretamente no console para o usuário.

O sistema também possui tratamento básico de erros durante o recebimento das mensagens. Caso ocorra alguma falha na comunicação, perda de conexão ou encerramento do servidor, o processo de leitura é interrompido automaticamente através do bloco catch, evitando que o programa apresente falhas inesperadas.

Enquanto a thread secundária permanece responsável pelo recebimento das mensagens, a thread principal continua executando o envio de

dados. O usuário pode digitar mensagens continuamente no console, criando uma comunicação dinâmica e interativa.

Antes de enviar cada mensagem, o sistema adiciona automaticamente algumas informações importantes:

- o horário atual da mensagem;
- o nome da estação de controle;
- o conteúdo digitado pelo usuário.

O horário é obtido utilizando a classe DateTime, no formato HH:mm, permitindo registrar o momento exato em que cada mensagem foi enviada. O resultado final segue um padrão semelhante ao utilizado em aplicativos profissionais de mensagens, por exemplo:

[14:35] Estacao01: Sistema operacional iniciado

Após a montagem da mensagem completa, o texto é convertido em bytes utilizando novamente a codificação UTF-8. Esses bytes são então enviados ao servidor através do método Write() do NetworkStream.

O funcionamento geral do cliente pode ser resumido nas seguintes etapas:

1. definição do IP e da porta do servidor;
2. conexão com o servidor TCP;
3. identificação do usuário;
4. criação da thread de recebimento;
5. envio contínuo de mensagens;
6. recebimento simultâneo de mensagens;
7. exibição das mensagens no console.

O programa demonstra diversos conceitos importantes relacionados à programação de redes e sistemas distribuídos, tais como:

- arquitetura cliente-servidor;
- utilização de sockets TCP;
- comunicação em tempo real;
- manipulação de fluxo de dados;
- codificação e conversão de mensagens;
- processamento simultâneo com threads;

- tratamento básico de erros de conexão;
- identificação de usuários em sistemas de comunicação.

Além disso, o projeto evidencia a importância da comunicação assíncrona em aplicações modernas, permitindo que múltiplas operações ocorram simultaneamente sem interromper a experiência do usuário.

O resultado final é um cliente funcional de chat em rede capaz de se comunicar com múltiplos usuários através de um servidor central, oferecendo troca de mensagens em tempo real, identificação dos participantes e funcionamento contínuo durante toda a conexão.

8. Apresentação do programa em funcionamento

8.1 Iniciando Servidor

Durante os testes realizados, o servidor da aplicação foi executado utilizando o terminal do .NET através do Visual Studio Code. O objetivo dos testes foi validar o funcionamento da comunicação em rede entre múltiplos clientes conectados simultaneamente ao servidor TCP.

- Inicialmente, foi acessada a pasta do projeto do servidor através do terminal. Em seguida, o servidor foi iniciado utilizando o comando: *dotnet run*.

O servidor foi iniciado corretamente, exibindo no console a seguinte mensagem:

Servidor iniciado...

Posteriormente, múltiplas centrais realizaram conexão com sucesso ao servidor. A cada nova conexão estabelecida, o sistema registrou automaticamente no terminal:

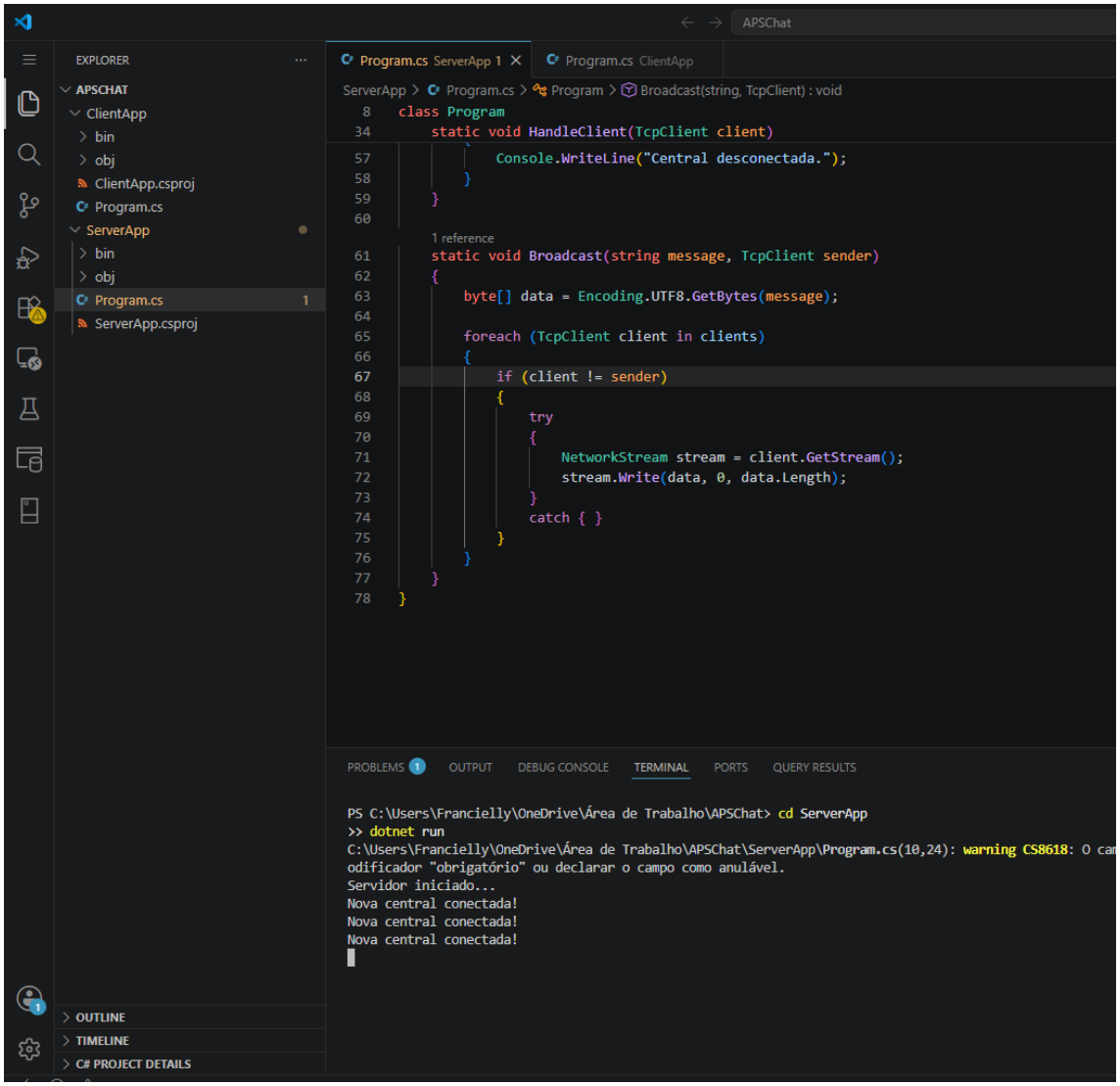
Nova central conectado!
Nova central conectado!
Nova central conectado!

Essas mensagens demonstram que o servidor conseguiu:

- iniciar corretamente;
- permanecer ativo aguardando conexões;
- aceitar múltiplos clientes simultaneamente;
- registrar as conexões realizadas;
- manter o funcionamento contínuo da aplicação.

Os testes comprovaram o funcionamento adequado da arquitetura cliente-servidor implementada utilizando sockets TCP/IP na linguagem C#.

Além disso, foi possível validar o uso de threads para tratamento simultâneo das conexões, permitindo que vários clientes utilizassem o sistema ao mesmo tempo sem interrupções na comunicação.



8.2 Inicialização do Client

Ao iniciar a aplicação, o cliente se conecta ao servidor TCP responsável pela comunicação entre os usuários.

Quando a conexão é realizada com sucesso, a seguinte mensagem é exibida:

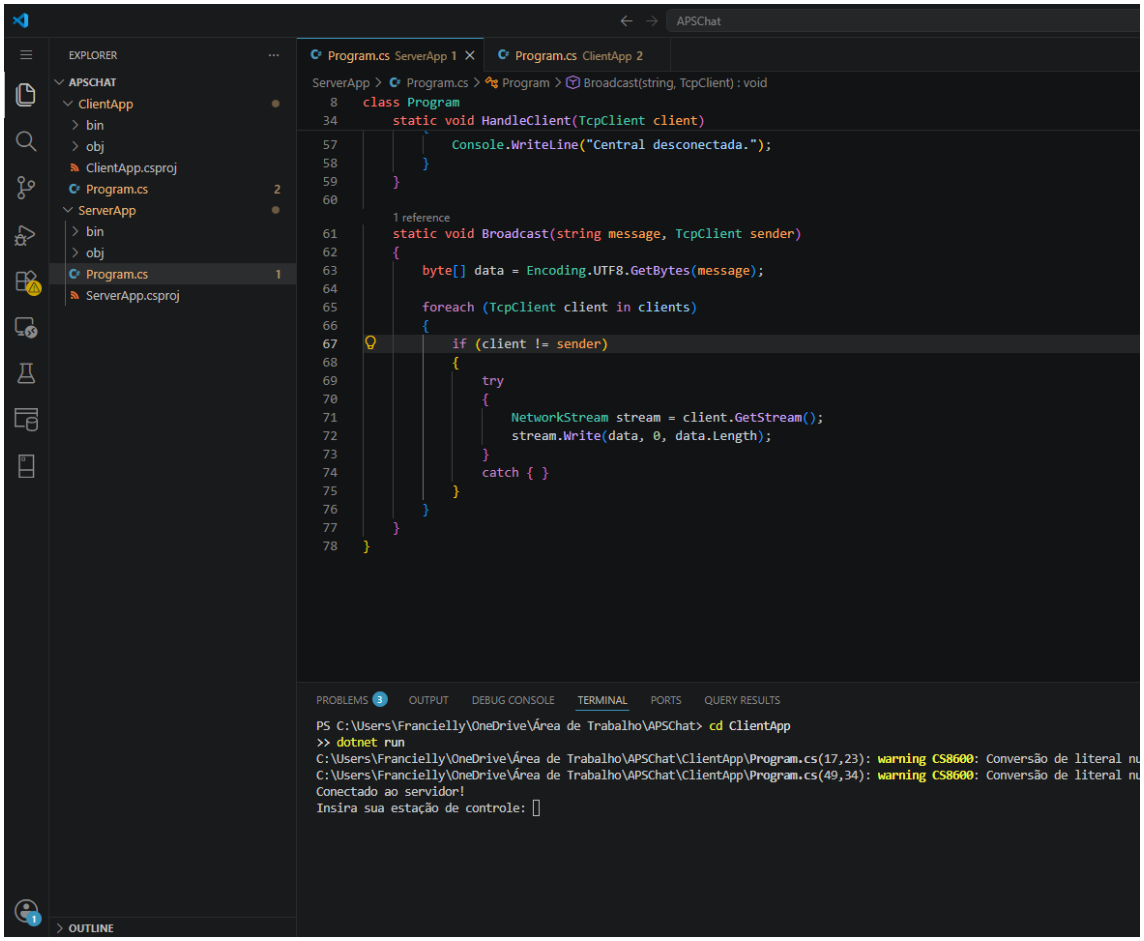
Conectado ao servidor!

Em seguida, o sistema solicita a identificação da estação de controle:

Insira sua estação de controle:

O usuário informa o nome da estação ou equipe responsável pelo monitoramento ambiental. Essa identificação é usada para registrar as mensagens enviadas no sistema.

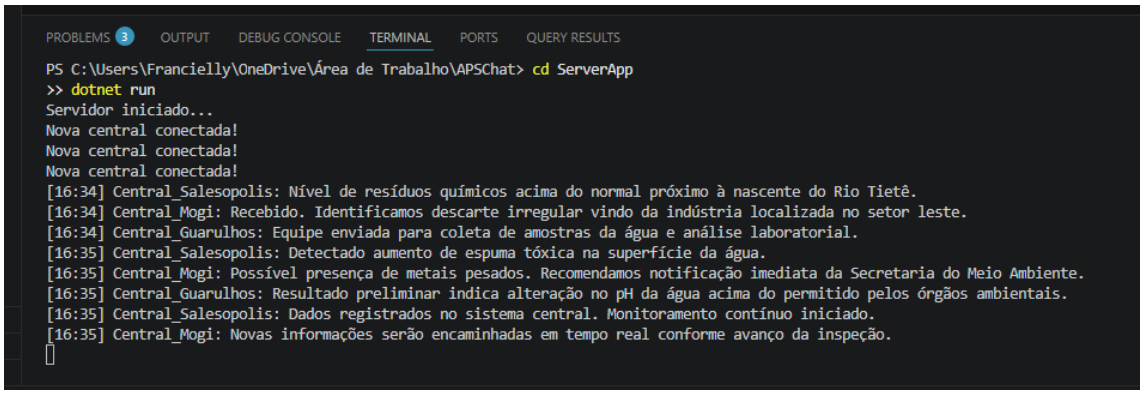
Após isso, o cliente permanece conectado e pode trocar mensagens em tempo real com os demais usuários da rede.



8.3 Exemplo de Comunicação Entre as Centrais de Monitoramento

O exemplo abaixo demonstra o funcionamento da aplicação em tempo real, apresentando a troca de mensagens entre diferentes estações de controle conectadas simultaneamente ao servidor principal.

As mensagens simulam a comunicação entre equipes responsáveis pelo monitoramento ambiental do Rio Tietê, conforme o cenário proposto na atividade prática supervisionada.


PROBLEMS 3
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
QUERY RESULTS
PS C:\Users\Francielly\OneDrive\Área de Trabalho\APSChat> cd ServerApp
>> dotnet run
Servidor iniciado...
Nova central conectada!
Nova central conectada!
Nova central conectada!
[16:34] Central_Salesopolis: Nível de resíduos químicos acima do normal próximo à nascente do Rio Tietê.
[16:34] Central_Mogi: Recebido. Identificamos descarte irregular vindo da indústria localizada no setor leste.
[16:34] Central_Guarulhos: Equipe enviada para coleta de amostras da água e análise laboratorial.
[16:35] Central_Salesopolis: Detectado aumento de espuma tóxica na superfície da água.
[16:35] Central_Mogi: Possível presença de metais pesados. Recomendamos notificação imediata da Secretaria do Meio Ambiente.
[16:35] Central_Guarulhos: Resultado preliminar indica alteração no pH da água acima do permitido pelos órgãos ambientais.
[16:35] Central_Salesopolis: Dados registrados no sistema central. Monitoramento contínuo iniciado.
[16:35] Central_Mogi: Novas informações serão encaminhadas em tempo real conforme avanço da inspeção.

REFERÊNCIAS BIBLIOGRÁFICAS

FOROUZAN, Behrouz A. Comunicação de Dados e Redes de Computadores. 4. ed. Porto Alegre: McGraw-Hill, 2010.

KUROSE, James F.; ROSS, Keith W. Redes de Computadores e a Internet: uma abordagem top-down. 8. ed. São Paulo: Pearson, 2021.

TANENBAUM, Andrew S.; WETHERALL, David. Redes de Computadores. 5. ed. São Paulo: Pearson, 2011.

MICROSOFT. Documentação oficial do .NET e C#. Disponível em: https://learn.microsoft.com/pt-br/dotnet/. Acesso em: 24 maio 2026.

MICROSOFT. TcpListener Class. Disponível em: https://learn.microsoft.com/pt-br/dotnet/api/system.net.sockets.tcplistener. Acesso em: 24 maio 2026.

MICROSOFT. TcpClient Class. Disponível em: https://learn.microsoft.com/pt-br/dotnet/api/system.net.sockets.tcpclient. Acesso em: 24 maio 2026.

Fichas APS

Letícia de Jesus Amorim

UNIP UNIVERSIDADE PAULISTA

FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

NOME: Letícia de Jesus Amorim TURMA: CC4P 68 RA: F35FjC-0

CURSO: Ciências da Computação CAMPUS: Paraiso SEMESTRE: 4° TURNO: Noturno

CÓDIGO DA ATIVIDADE: SEMESTRE: 4° semestre ANO GRADE:

DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	TOTAL DE HORAS	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)	ASSINATURA DO PROFESSOR
01/04	Planejamento com o grupo			4H	
03/04	Planejamento da ideia do código			6H	
04/04	Escrita da base do código			8H	
06/04	Pesquisa sobre Protocolos			4H	
08/04	Pesquisa sobre Fundamentos da comunicação			3H	
13/04	Pesquisa sobre tecnologias de comunicação			3H	
14/04	Pesquisa sobre conexões			2h	
16/04	implementação das threads			4h	
20/04	Inicio da documentação			2h	
21/04	aprofundamento das pesquisas			4H	
25/04	continuação da documentação			8H	
02/05	correção de erros no código			6H	
03/04	correção de erros na documentação			6h	
16/04	revisão geral			3H	
23/04	reunião ára considerações finais e entrega			5h	

(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

TOTAL DE HORAS ATRIBUÍDAS: 68 Horas

AVALIAÇÃO:

Aprovado ou Reprovado

NOTA:

DATA: 23 / 05 / 2026

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO

Mateus Ruiz

UNIP UNIVERSIDADE PAULISTA

FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

NOME: Mateus Ruiz TURMA: CC4P 68 RA: G7985I2

CURSO: Ciências da computação CAMPUS: Paraiso SEMESTRE: 4° TURNO: Noturno

CÓDIGO DA ATIVIDADE: SEMESTRE: 4° semestre ANO GRADE:

DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	TOTAL DE HORAS	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)	ASSINATURA DO PROFESSOR
01/04	Planejamento com o Grupo			4H	
03/04	Planejamento da ideia do código		MATEUS	6H	
20/04	Iniciação da documentação			2h	
02/05	correção de erros no codigo			6H	

(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

TOTAL DE HORAS ATRIBUÍDAS: 18 Horas

AVALIAÇÃO:

Aprovado ou Reprovado

NOTA:

DATA: 23 / 05 / 2026

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO

João Lucas Antunes Modena

UNIP
UNIVERSIDADE PAULISTA

FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

NOME: João Lucas Antunes Modena TURMA: CC5P 68 RA: R096AD4

CURSO: Ciências da Computação CAMPUS: Paraiso SEMESTRE: 5° TURNO: Noturno

CÓDIGO DA ATIVIDADE: SEMESTRE: 5° semesatre ANO GRADE:

DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	TOTAL DE HORAS	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)	ASSINATURA DO PROFESSOR
01/04	Planejamento com o grupo			4H	
03/04	Planejamento da ideia do código			6h	
04/04	pesquisa sobre protocolo			4H	
21/04	Aprofundamento nas pesuisas			4H	

(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

TOTAL DE HORAS ATRIBUÍDAS: 18 horas

AVALIAÇÃO:
Aprovado ou Reprovado

NOTA:

DATA: 23 / 05 / 2026

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO